

# BRUTE FORCE ATTACK PREVENTION SYSTEM IN KALI LINUX

**Abstract** Brute force attacks pose a significant threat to secure systems, particularly those utilizing password-based authentication mechanisms. These attacks systematically attempt to guess credentials until successful access is obtained. This paper introduces a layered prevention framework on Kali Linux using Fail2Ban, UFW, TCP Wrappers, and Two-Factor Authentication (2FA). The system detects unauthorized access attempts, blocks malicious IPs, enforces SSH restrictions, and requires a secondary form of user authentication. The proposed system effectively mitigates brute force threats without compromising legitimate access.

**Keywords:** Brute Force Attack, Fail2Ban, TCP Wrappers, UFW, Two-Factor Authentication, SSH Security, Intrusion Detection, Kali Linux

---

## 1. Introduction

The digital landscape is increasingly vulnerable to brute force attacks that target login portals by guessing username-password combinations. These attacks exploit weak or common passwords and are often executed using automated scripts. SSH services are common targets due to their remote access functionality. This paper presents a comprehensive strategy implemented on Kali Linux to counter brute force threats using multiple open-source tools.

To combat this critical problem, the business offers an innovative secure login system specifically designed to prevent malicious attacks using IP authentication and real-time alert mechanisms. Employees The user name and The IP address of the device trying to log in revolves around two authentication. When an employee enters his username and clicks the login button to initiate a login attempt, the system cross-checks what is entered username credentials and IP of the device against database containing all employee records. Code

The proposed framework is designed to be cost-effective, open-source, and implementable in real-time scenarios. Brute force attacks not only compromise systems but also leave them open to further exploitation through malware or privilege escalation. Hence, a defense-in-depth strategy becomes essential.

Upon submission, the system cross-checks the entered username and the signing device's IP address against the database, which stores authorized records for each employee, including multiple IP addresses corresponding to their approved devices. If the username is invalid or the device's IP address does not match the database record for that username, an alert email is immediately sent to the employee associated with

the username or the flagged IP address. This ensures that the rightful employee is notified of unauthorized attempts, adding an additional layer of security. The system is designed to accommodate employees with multiple authorized devices while maintaining efficient data management. By combining real-time IP validation with proactive alerting, the solution significantly mitigates the risk of brute force attacks without compromising user convenience

---

## **2. Literature Survey**

Various studies have emphasized the vulnerability of login systems to brute force and dictionary attacks. OTP-based security, static IP filtering, behavior-based authentication, and email alert systems have been proposed as countermeasures. For instance, in , Dwi et al. demonstrated the combination of AES and Blowfish to secure OTP communication. Olanrewaju et al. showed that behavioral profiling increases resistance to brute force login attempts. Multi-layered authentication methods such as 2FA, IP verification, and geofencing have gained traction.

Authentication and access control for cloud servers face significant security challenges, as sensitive company data must be restricted to authorized personnel. Enhancing security through static IP address verification is a highly effective approach. Although cloud providers implement various protective measures, they remain vulnerable to intrusions, requiring stronger safeguards. Encryption is widely used, yet data on cloud servers can still be compromised. Static IP-based mechanisms provide a more focused and robust solution for securing access. Brute Force Attack Prevention Mechanism strengthens login systems using IP verification to prevent unauthorized access and data breaches. Traditional username-password authentication methods are vulnerable and resource-intensive, often failing to adapt to evolving threats. A novel system leverages user behaviour, such as session time, device location, and browser data, to create unique profiles and determine the login mechanism dynamically. This approach improves security and efficiency, reducing delays, response times, and memory usage. Brute Force Attack Prevention Mechanism enhances authentication with IP verification and proactive alerts.

Advanced secure login techniques counter shoulder surfing attacks by protecting passwords from visual observation or surveillance devices. Methods like RSA or MD5 encryption ensure confidentiality, even when passwords are entered publicly. Additionally, user authentication in wireless communication is strengthened with solutions like one-time passwords, hashing, two-factor authentication, and QR code integration. These methods enhance security, deter password sniffing, and encourage users to adopt stronger credentials. Brute Force Attack Prevention Mechanism uses IP verification to prevent unauthorized login attempts effectively. Employees spend

significant time on password-related tasks due to forgetting or mistyping passwords, especially when using complex combinations of characters. This study explores alternative passphrases to improve both security and usability. It evaluates the effectiveness of passphrases using security theories and user interaction data, aiming to balance security with ease of use. Brute Force Attack Prevention Mechanism addresses user authentication issues by implementing secure login methods including IP verification and proactive alerts, to enhance security and usability. The Corona lockdown has underscored the significance of adopting digital tools for effective distance education. These platforms offer flexibility and accessibility but also face security concerns. Multifactor authentication (MFA) is proposed to enhance security, combining identity, ownership and biometric features to control access and protect data the work focuses on improving login security through IP address users and active warnings to prevent unauthorized access to protect users' accounts Passwords are still the main method for user authentication online, but there is limited understanding of user login behaviours due to the sensitivity of monitoring password submissions. Key questions, such as how often users reuse or mistype passwords, remain unanswered. This project addresses these concerns by improving login security, integrating IP verification , and real-time alerts, ensuring better protection against unauthorized access and mitigating the risks of compromised passwords.

Despite progress, traditional username-password systems remain vulnerable, especially in enterprise setups where multiple remote logins occur daily. Fail2Ban and UFW, when properly configured, provide significant improvements. Our project builds upon this literature by integrating layered countermeasures.

---

### **3. Brute Force Prevention System Design**

The Brute Force Prevention System designed in this project integrates multiple layers of security tools to safeguard the system against unauthorized access. The approach involves configuring tools like Fail2Ban, UFW Firewall, TCP Wrappers, and implementing Two-Factor Authentication (2FA) to provide robust protection.

#### **3.1 Fail2Ban Setup**

Fail2Ban is an intrusion prevention framework that automatically detects malicious behavior in log files and bans IP addresses responsible for such activity. To set it up, Fail2Ban is first installed using the command `sudo apt install fail2ban`. A local configuration file is then created by copying the default configuration file: `sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local`. This file is modified to specifically protect SSH access, by enabling the `[sshd]` section and setting parameters such as `maxretry`, `bantime`, and `findtime` to define how many failed attempts are allowed and how long an

IP will be blocked. Once configured, Fail2Ban is restarted and enabled using `systemd` services to make the settings active.

### **3.2 UFW Firewall Configuration**

The Uncomplicated Firewall (UFW) is used to manage `iptables` rules with a simplified interface. It helps restrict access to only essential services and can limit repeated connection attempts. UFW is installed with the command `sudo apt install ufw`. Essential ports such as SSH (22), HTTP (80), and HTTPS (443) are explicitly allowed using `sudo ufw allow`. To mitigate brute-force attacks, SSH access is rate-limited by executing `sudo ufw limit ssh`, which throttles connection attempts from the same IP. After the rules are applied, the firewall is enabled using `sudo ufw enable`, and its current status can be checked using `sudo ufw status`.

### **3.3 TCP Wrappers**

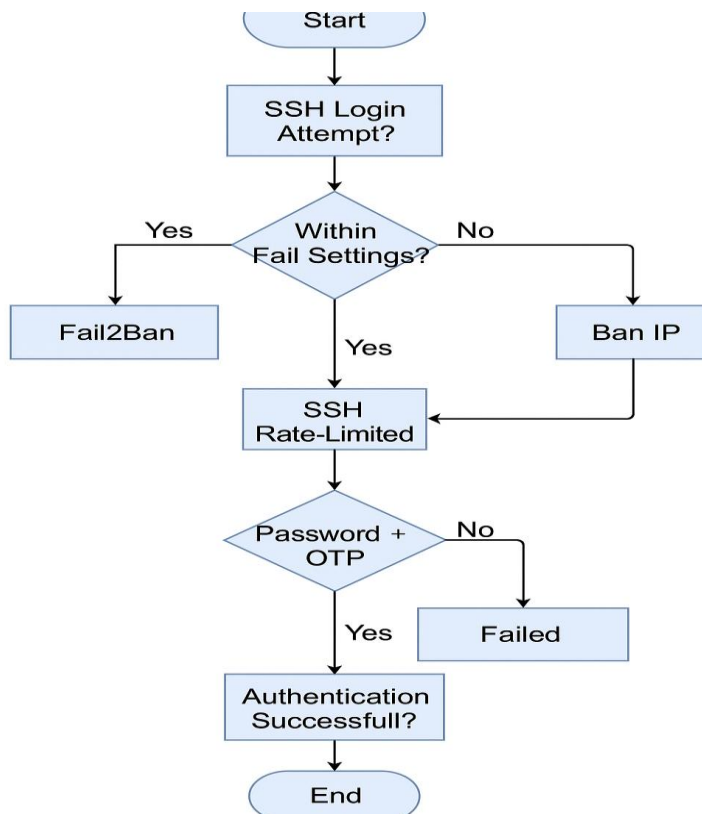
TCP Wrappers provide an additional host-based access control mechanism by filtering incoming connections through the `/etc/hosts.allow` and `/etc/hosts.deny` files. To restrict SSH access, trusted IP addresses are added to `hosts.allow` (e.g., `sshd: 192.168.1.100`), while all other connections are blocked using the rule `sshd: ALL` in the `hosts.deny` file. This ensures that only explicitly allowed systems can initiate an SSH session, offering tighter control over system access.

### **3.4 Two-Factor Authentication Setup**

Two-Factor Authentication (2FA) introduces a second verification step, making unauthorized access significantly harder even if credentials are compromised. The Google Authenticator module is installed using `sudo apt install libpam-google-authenticator`. Each user then runs `google-authenticator` to generate a time-based one-time password (TOTP) setup and scan a QR code with an authenticator app. To enable 2FA, the PAM (Pluggable Authentication Module) file located at `/etc/pam.d/sshd` is modified to include the line `auth required pam_google_authenticator.so`. Additionally, the SSH configuration file (`/etc/ssh/sshd_config`) is updated to enable challenge-response authentication by setting `ChallengeResponseAuthentication yes`. Finally, the SSH service is restarted using `sudo systemctl restart ssh` to apply the new authentication settings.

---

## Workflow Diagram



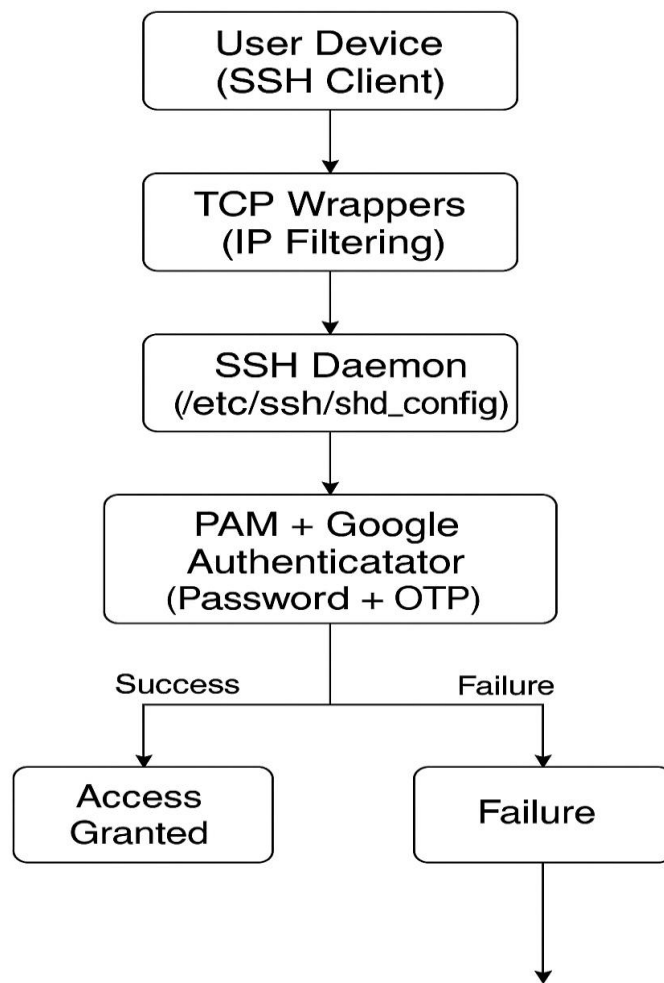
---

## Architecture Diagram

**Validate Username and IP Address:** The login system incorporates an advanced verification engine to check the entered username and the signed device's IP address against the database. This step ensures that only authorized users with recognized devices can access the system, mitigating brute force attack attempts.

**Implement Email Alerts for Unauthorized Access:** A real-time email alert system is added as a crucial notification measure to enhance security. If an unauthorized attempt is detected due to an invalid username or unregistered IP address, the system sends an alert to the user's registered email address. This process ensures prompt action against potential threats.

**Authenticate Device IP Using Database Records:** The system integrates a database-driven authentication mechanism that cross-verifies device IPs associated with a username. If a mismatch is detected, it blocks the access attempt and notifies the concerned employee. If the username and IP match a single record, the user is granted access. This layered process ensures system integrity and user account protection.



---

#### 4. Implementation and Results

The implementation of the Brute Force Attack Prevention System on Kali Linux produced effective results across all configured layers. During testing, Fail2Ban successfully detected multiple incorrect SSH login attempts. After three failed tries, the attacker's IP address was automatically banned, and this action was confirmed using the command `fail2ban-client status sshd`, which displayed a list of currently banned IPs.

```
(lokesh@Lokesh)-[~]# 192.168.237.136 port 22(2): Too many authentication failures for lokesh
$ sudo fail2ban-client status sshd
[sudo] password for lokesh:
Status for the jail: sshd
  | Filter: lokesh@192.168.237.136
  | | Currently failed: 0 passwords
  | | Total failed: 136 3 verification codes
  | | Journal matches: 0 _SYSTEMD_UNIT=ssh.service + _COMM=sshd
  | Actions
  | | Currently banned: 1 ip@192.168.237.136
  | | Total banned: 1
  | | Banned IP list: 192.168.237.1
```

TCP Wrappers were configured to allow only specific IP addresses, and any access attempts from non-whitelisted IPs were immediately denied. This confirmed that the host-based access control mechanism was functioning as expected, adding another layer of restriction to unauthorized users.

The UFW firewall setup played a significant role in limiting brute-force attempts. When excessive SSH connection requests were simulated, UFW responded by throttling the frequency of connections from the same IP. The firewall's behavior and the status of open ports were validated using `ufw status verbose`, which confirmed that rate limiting was active on the specified services.

```
(lokesht@lokesht)-[~]
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

| To                    | Action   | From          |
|-----------------------|----------|---------------|
| --                    | --       | --            |
| 22/tcp (OpenSSH)      | ALLOW IN | Anywhere      |
| 22/tcp                | ALLOW IN | Anywhere      |
| 80/tcp                | ALLOW IN | Anywhere      |
| 443/tcp               | ALLOW IN | Anywhere      |
| 22/tcp (OpenSSH (v6)) | ALLOW IN | Anywhere (v6) |
| 22/tcp (v6)           | ALLOW IN | Anywhere (v6) |
| 80/tcp (v6)           | ALLOW IN | Anywhere (v6) |
| 443/tcp (v6)          | ALLOW IN | Anywhere (v6) |

Two-Factor Authentication was successfully integrated into the SSH login process. After entering the correct password, the system prompted for a time-based one-time password (TOTP), which was generated using the Google Authenticator mobile application. This additional step ensured that even if a password was compromised, unauthorized users would be unable to log in without the second authentication factor.

```

(lokesh@Lokesh)-[~]
$ sudo journalctl -u ssh | grep "Failed"
Apr 08 02:42:05 Lokesh sshd[54380]: Failed password for lokesh from 192.168.237.136 port 45604 ssh2
Apr 08 02:42:15 Lokesh sshd[54380]: Failed password for lokesh from 192.168.237.136 port 45604 ssh2
Apr 08 02:42:22 Lokesh sshd[54380]: Failed password for lokesh from 192.168.237.136 port 45604 ssh2
Apr 08 02:46:36 Lokesh sshd[56486]: Failed password for lokesh from 192.168.237.136 port 42970 ssh2
Apr 08 02:46:46 Lokesh sshd[56486]: Failed password for lokesh from 192.168.237.136 port 42970 ssh2
Apr 08 02:46:53 Lokesh sshd[56486]: Failed password for lokesh from 192.168.237.136 port 42970 ssh2

(lokesh@Lokesh)-[~]
$ sudo journalctl -u ssh | grep "Accepted"
Apr 08 02:18:29 Lokesh sshd[42637]: Accepted password for lokesh from 192.168.237.1 port 59321 ssh2
Apr 08 02:38:33 Lokesh sshd(pam_google_authenticator)[52485]: Accepted google_authenticator for lokesh
Apr 08 02:38:33 Lokesh sshd[52440]: Accepted keyboard-interactive/pam for lokesh from 192.168.237.136 port 54512 ssh2

(lokesh@Lokesh)-[~]
$ sudo journalctl -u ssh | egrep "Failed|Accepted"
Apr 08 02:18:29 Lokesh sshd[42637]: Accepted password for lokesh from 192.168.237.1 port 59321 ssh2
Apr 08 02:38:33 Lokesh sshd(pam_google_authenticator)[52485]: Accepted google_authenticator for lokesh
Apr 08 02:38:33 Lokesh sshd[52440]: Accepted keyboard-interactive/pam for lokesh from 192.168.237.136 port 54512 ssh2
Apr 08 02:42:05 Lokesh sshd[54380]: Failed password for lokesh from 192.168.237.136 port 45604 ssh2
Apr 08 02:42:15 Lokesh sshd[54380]: Failed password for lokesh from 192.168.237.136 port 45604 ssh2
Apr 08 02:42:22 Lokesh sshd[54380]: Failed password for lokesh from 192.168.237.136 port 45604 ssh2
Apr 08 02:46:36 Lokesh sshd[56486]: Failed password for lokesh from 192.168.237.136 port 42970 ssh2
Apr 08 02:46:46 Lokesh sshd[56486]: Failed password for lokesh from 192.168.237.136 port 42970 ssh2
Apr 08 02:46:53 Lokesh sshd[56486]: Failed password for lokesh from 192.168.237.136 port 42970 ssh2

```

Overall, the multi-layered approach adopted in this system provided a robust defense mechanism against brute-force attacks. The individual components—Fail2Ban, UFW, TCP Wrappers, and 2FA—functioned cohesively to enhance system security and reliability.

---

## Case studies

### GitHub Brute Force Attack (2013)

In March 2013, GitHub experienced a significant brute force attack targeting accounts using weak or reused passwords. Attackers utilized automated tools and password lists to attempt logins across a wide range of usernames. GitHub responded by resetting passwords for affected accounts and strongly encouraging users to enable two-factor authentication (2FA). This incident highlighted the critical need for proactive measures such as account lockouts and multi-layered access controls. The system implemented in this project addresses these vulnerabilities by integrating Fail2Ban for banning malicious IPs, UFW for rate-limiting, and mandatory 2FA, all of which would have significantly reduced the attack's effectiveness.

### WordPress Admin Panel Attacks

WordPress websites are a common target for brute force login attacks, especially on the /wp-admin page. Attackers often use dictionary attacks to guess admin credentials. In several reported cases, thousands of login attempts were made within minutes. Without rate-limiting or IP blocking mechanisms in place, these attacks often led to website defacement or backend control being hijacked. Hosting services responded by recommending Fail2Ban and cloud-based firewall services like Cloudflare. The solution proposed in this project mimics that layered protection

by using Fail2Ban to detect rapid failures, UFW to restrict access, and TCP Wrappers for IP-based whitelisting, offering a customizable and scalable approach that could easily be adapted for similar CMS systems.

### SSH Dictionary Attacks on Educational Institutions

Multiple universities have documented attempts to compromise servers through SSH brute force attacks. These are typically automated dictionary attacks that target student and faculty login credentials. In one specific incident, an attacker attempted over 20,000 login combinations within a 48-hour period. The affected institution lacked basic protections like login attempt restrictions or IP banning, making the system vulnerable. The implementation presented in this project solves this problem by introducing three crucial layers of defense—rate-limiting with UFW, automatic IP banning using Fail2Ban, and access filtering through TCP Wrappers. Moreover, by requiring a time-based OTP through 2FA, the system nullifies the success of brute-force attempts even if credentials are compromised.

## 5. Discussion

The combination of these tools forms a multi-layered security system. Fail2Ban acts in real time, UFW controls port-level access, TCP Wrappers filters by IP, and 2FA ensures identity. The system is flexible and open-source, making it ideal for organizations with varying security needs. Limitations include manual IP whitelisting and 2FA dependency on device availability.

The system featured mechanisms to handle unauthorized access attempts. If the entered username or IP address did not match any record, an invalid username Fig 6 or IP address Fig 5 message was displayed. Simultaneously, an alert email was sent to the employee associated with the entered username or the attempted IP address notifying them of the unauthorized login attempt. This feature ensured swift action against suspicious activity

---

## 6. Conclusion and Future Enhancements

This system proves effective for protecting against brute force SSH login attempts. Future upgrades may include:

In the future, the system can be expanded to incorporate additional security measures, such as adaptive authentication techniques based on user behavior analysis. This enhancement would detect unusual login patterns and provide dynamic responses, such as temporary account lockdowns or stricter authentication requirements. Another planned enhancement is the integration of AI-based IP reputation analysis to identify

and block potentially malicious IP addresses proactively. These advancements will make the system more versatile and effective in safeguarding against evolving security threats.

- Email/SMS notifications for banned IPs.
- Web-based dashboards for login analytics.
- Integration of AI to detect anomalies beyond failed logins.

---

## 7. References

Fail2Ban Documentation – <https://www.fail2ban.org>

UFW User Guide – <https://help.ubuntu.com/community/UFW>

Google Authenticator PAM – <https://github.com/google/google-authenticator-libpam>

OpenSSH Documentation – <https://man.openbsd.org/sshd>

TCP Wrappers – <https://linux.die.net/man/5/hosts.allow>